

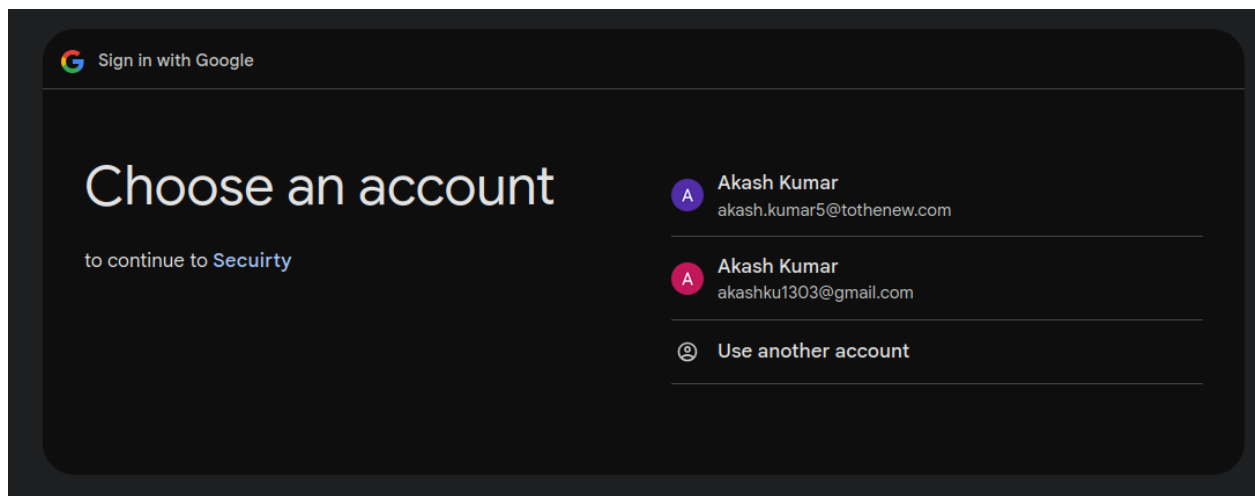
# Assignment - Spring Security - Part 2

Q1) Implement oauth 2 using spring security and authenticate a user which is saved in database using spring data jpa.



**Welcome**

[Login with Google](#)





http://localhost:8080

# Welcome

Hello, **Akash Kumar**!

Your email: **akashku1303@gmail.com**

[Logout](#)

4 rows in set (0.00 sec)

```
mysql> select * from user;
```

| id | email                 | name        | provider | provider_id           |
|----|-----------------------|-------------|----------|-----------------------|
| 1  | akashku1303@gmail.com | Akash Kumar | google   | 117601416879124959792 |

1 row in set (0.01 sec)

```
mysql>
```

Q2)Grant different Roles to different users and make sure that only authorized users of a given type can access the resource.

---

Hello USER: akashku1303@gmail.com

## Login with OAuth 2.0

You have been signed out

[Google](#)

---

Hello ADMIN: akash.kumar5@tothenew.com

```
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> select * from user;
```

| id | email                     | name        | provider | provider_id           | role  |
|----|---------------------------|-------------|----------|-----------------------|-------|
| 2  | akashku1303@gmail.com     | Akash Kumar | google   | 117601416879124959792 | USER  |
| 3  | akash.kumar5@tothenew.com | Akash Kumar | google   | 106248813046807265532 | ADMIN |

2 rows in set (0.00 sec)

```
mysql>
```

```

import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api")
public class SampleController {

    @GetMapping("/user/info")
    public String userAccess(OAuth2AuthenticationToken token) {
        CustomUserPrincipal user = (CustomUserPrincipal) token.getPrincipal();
        return "Hello USER: " + user.getEmail();
    }

    @GetMapping("/admin/dashboard")
    public String adminAccess(OAuth2AuthenticationToken token) {
        CustomUserPrincipal user = (CustomUserPrincipal) token.getPrincipal();
        return "Hello ADMIN: " + user.getEmail();
    }
}

```

```

public class SecurityConfig {

    private final CustomOAuth2UserService customOAuth2UserService;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .oauth2Login( OAuth2LoginConfigurer<HttpSecurity> oauth -> oauth
                .userInfoEndpoint( UserInfoEndpointConfig userInfo -> userInfo
                    .userService(customOAuth2UserService)
                )
            );

        return http.build();
    }
}

```

```
import java.util.HashSet;

@Service
@RequiredArgsConstructor
public class CustomOAuth2UserService extends DefaultOAuth2UserService {

    private final UserRepository userRepository;

    @Override // 1 usage
    public OAuth2User loadUser(OAuth2UserRequest userRequest) throws OAuth2AuthenticationException {
        OAuth2User oAuth2User = super.loadUser(userRequest);
        String email = oAuth2User.getAttribute( name: "email");

        User user = userRepository.findByEmail(email).orElseGet(() -> {
            return userRepository.save(User.builder()
                .email(email)
                .name(oAuth2User.getAttribute( name: "name"))
                .provider(userRequest.getClientRegistration().getRegistrationId())
                .providerId(oAuth2User.getName())
                .role(Role.USER)
                .build());
        });

        return new CustomUserPrincipal(user, oAuth2User.getAttributes());
    }
}
```